BigQuery Basics: A Hands-On Lab Guide

Level: Beginner ·

Overview

BigQuery is Google Cloud's serverless data warehouse. You store data in tables and analyze it with standard SQL, without managing servers. You pay (or use free quota) based on how much data you **store** and how much data your queries **scan**.

By the end of this guide you will be able to:

1. Set up a free BigQuery environment.
2. Navigate the BigQuery console and explore public datasets.
3. Write SQL queries to filter, sort, and aggregate data.
4. Create your own dataset and load a CSV file into a table.
5. Join tables, use CTEs, and save query results.
6. Estimate query cost and write cheaper, faster queries.

Prerequisites: a Google account and a web browser. Some SQL familiarity helps but isn't required; each query is explained.

Key terms to know before you start

Term	**Meaning**
Project	The top-level container in Google Cloud. Billing and permissions live here.
Dataset	A folder inside a project that holds tables and views.
Table	Rows and columns of data, with a defined schema.
Schema	The list of column names and their data types (STRING, INT64, DATE, etc.).
Job	Any action BigQuery runs for you: a query, a load, an export.

Term	Meaning
Bytes processed	How much data a query scans. This is what drives query cost.

Tables are referenced as `project.dataset.table`, for example `bigquery-public-data.usa_names.usa_1910_2013`.

Lab 1: Set Up Your Environment (If project exists pls skip this step)

Goal: Get a working BigQuery project at no cost.

Step 1.1 — Open the console. Go to https://console.cloud.google.com/bigquery and sign in with your Google account. Accept the terms of service if prompted.

Step 1.2 — Create a project. If prompted, click **Create Project** (or use the project picker at the top of the page → **New Project**). Name it something like `bq-learning-lab` and click **Create**. Make sure this project is selected in the project picker.

Step 1.3 — Confirm you're in the Sandbox. If you didn't add a billing account, you'll see a "Sandbox" banner at the top. The Sandbox is free and has some limits:

- Around 10 GB of storage and 1 TB of query processing per month free.
- Tables you create expire automatically after 60 days.
- Some features are unavailable, such as DML statements (`INSERT`, `UPDATE`, `DELETE`) and streaming inserts.

That's fine for this guide. Limits can change, so check Google's BigQuery Sandbox documentation for current details.

Step 1.4 — Enable the API (if asked). If you see a message about the BigQuery API not being enabled, click **Enable**.

✅ **Checkpoint:** You see the BigQuery Studio page with an **Explorer** panel on the left and a query editor in the middle.

Lab 3: Your First SQL Queries

Goal: Use `SELECT`, `WHERE`, `ORDER BY`, and `LIMIT`.

Step 3.1 — Run a basic query. Paste this into the editor:

```
SELECT name, gender, number

FROM `bigquery-public-data.usa_names.usa_1910_2013`

LIMIT 10;
```

Before running, look at the top right of the editor. The **validator** shows a green check and a message like "This query will process X MB when run." Click **Run**.

Note the backticks (`` ` ``) around the table name. They're required when the name contains hyphens.

Step 3.2 — Filter with WHERE.

```
SELECT name, year, number

FROM `bigquery-public-data.usa_names.usa_1910_2013`

WHERE state = 'TX'

  AND year = 2000

  AND gender = 'F';
```

Step 3.3 — Sort with ORDER BY. Find the most common girls' names in Texas in 2000:

```
SELECT name, number

FROM `bigquery-public-data.usa_names.usa_1910_2013`

WHERE state = 'TX' AND year = 2000 AND gender = 'F'

ORDER BY number DESC

LIMIT 10;
```

Step 3.4 — Pattern matching and lists.

SELECT DISTINCT name

FROM `bigquery-public-data.usa_names.usa_1910_2013`

WHERE name LIKE 'Ar%'

  AND state IN ('CA', 'NY')

ORDER BY name;

`LIKE 'Ar%'` matches names starting with "Ar". `DISTINCT` removes duplicates.

Step 3.5 — Create computed columns.

SELECT

  name,

  year,

  number,

  UPPER(name) AS name_upper,

  LENGTH(name) AS name_length

FROM `bigquery-public-data.usa_names.usa_1910_2013`

WHERE state = 'WA' AND year = 1990

LIMIT 20;

✏️ **Exercise 3:** Write a query that lists the 5 most common boys' names in California in 1985.

Solution SELECT name, number

FROM `bigquery-public-data.usa_names.usa_1910_2013`

WHERE state = 'CA' AND year = 1985 AND gender = 'M'

ORDER BY number DESC

LIMIT 5;

> **Important:** `LIMIT` reduces the rows *returned*, not the data *scanned*. On a normal table, BigQuery still reads every column you selected. You'll see why this matters in Lab 7.

Lab 4: Aggregations and GROUP BY

Goal: Summarize data with `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP BY`, and `HAVING`.

Step 4.1 — Total across all rows.

SELECT SUM(number) AS total_babies

FROM `bigquery-public-data.usa_names.usa_1910_2013`;

Step 4.2 — Group by one column. Most popular names of all time:

SELECT name, SUM(number) AS total

FROM `bigquery-public-data.usa_names.usa_1910_2013`

GROUP BY name

ORDER BY total DESC

LIMIT 10;

Every column in `SELECT` that isn't inside an aggregate function must appear in `GROUP BY`.

Step 4.3 — Group by several columns. Births recorded per year by gender:

SELECT year, gender, SUM(number) AS total

FROM `bigquery-public-data.usa_names.usa_1910_2013`

GROUP BY year, gender

ORDER BY year, gender;

After it runs, click **Chart** (or **Explore data**) in the results pane to visualize it as a line chart.

Step 4.4 — Filter groups with HAVING. `WHERE` filters rows *before* grouping; `HAVING` filters groups *after*.

SELECT name, COUNT(DISTINCT state) AS states_used_in

FROM `bigquery-public-data.usa_names.usa_1910_2013`

WHERE year = 2010

GROUP BY name

HAVING states_used_in = 50

ORDER BY name;

Step 4.5 — Several aggregates at once.

SELECT

  corpus,

  COUNT(*) AS distinct_words,

  SUM(word_count) AS total_words,

  MAX(word_count) AS most_frequent_word_count

FROM `bigquery-public-data.samples.shakespeare`

GROUP BY corpus

ORDER BY total_words DESC;

✏️ **Exercise 4:** Which state recorded the most babies in the dataset overall? Show the state and the total.

Solution SELECT state, SUM(number) AS total

FROM `bigquery-public-data.usa_names.usa_1910_2013`

GROUP BY state

```
ORDER BY total DESC

LIMIT 1;
```

Lab 5: Create Your Own Dataset and Load Data

Goal: Store your own data in BigQuery.

Step 5.1 — Create a dataset.

1. In Explorer, click the three-dot menu next to your project → **Create dataset**.
2. **Dataset ID:** `lab_data`
3. **Location type:** pick a region or multi-region (e.g., `US` or `asia-south1`). Remember it: datasets you join together must be in the same location. Public datasets are mostly in `US`, so `US` is the easiest choice for this guide.
4. Click **Create dataset**.

Step 5.2 — Prepare a CSV file. Create a file named `products.csv` on your computer with this content:

```
product_id,product_name,category,price,launch_date

1,Notebook,Stationery,3.50,2024-01-15

2,Pen Set,Stationery,7.25,2024-02-01

3,Desk Lamp,Electronics,24.99,2024-03-10

4,USB Cable,Electronics,5.99,2024-03-22

5,Water Bottle,Kitchen,12.00,2024-04-05

6,Coffee Mug,Kitchen,8.50,2024-04-18

7,Headphones,Electronics,49.99,2024-05-02

8,Sticky Notes,Stationery,2.75,2024-05-20
```

Step 5.3 — Load the CSV into a table.

1. Click the three-dot menu next to `lab_data` → **Create table**.
2. **Create table from:** Upload → select `products.csv`. **File format:** CSV.
3. **Table:** `products`
4. **Schema:** tick **Auto detect**.
5. Under **Advanced options**, set **Header rows to skip** to `1` (auto detect often handles this, but setting it is safer).
6. Click **Create table**.

Step 5.4 — Verify. Click the new table and check the **Schema** tab. Confirm `price` is `FLOAT` and `launch_date` is `DATE`. Then query it:

```
SELECT *

FROM `lab_data.products`

ORDER BY price DESC;
```

You can omit the project ID when the table is in your currently selected project.

Step 5.5 — Create a table with SQL (DDL). You can also create tables from query results:

```
CREATE OR REPLACE TABLE `lab_data.electronics` AS

SELECT product_id, product_name, price

FROM `lab_data.products`

WHERE category = 'Electronics';
```

Step 5.6 — Create a view. A view is a saved query that behaves like a table but stores no data:

```
CREATE OR REPLACE VIEW `lab_data.cheap_products` AS

SELECT product_name, price

FROM `lab_data.products`

WHERE price < 10;
```

Then run `SELECT * FROM lab_data.cheap_products;`

✅ **Checkpoint:** Your `lab_data` dataset contains a `products` table, an `electronics` table, and a `cheap_products` view.

> **Sandbox note:** `INSERT`, `UPDATE`, and `DELETE` aren't available in the Sandbox. To change data, use `CREATE OR REPLACE TABLE ... AS SELECT` with the new rows, or enable billing.

Lab 6: Joins, CTEs, and Saving Results

Goal: Combine tables and organize longer queries.

We'll use the Austin bike-share public data: `bigquery-public-data.austin_bikeshare.bikeshare_trips` and `bigquery-public-data.austin_bikeshare.bikeshare_stations`. Check both schemas in Explorer first.

Step 6.1 — INNER JOIN. Show trips alongside the status of their starting station:

```
SELECT
  t.trip_id,
  t.start_station_name,
  t.duration_minutes,
  s.status AS station_status
FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips` AS t
JOIN `bigquery-public-data.austin_bikeshare.bikeshare_stations` AS s
  ON CAST(t.start_station_id AS STRING) = CAST(s.station_id AS STRING)
LIMIT 20;
```

`AS t` and `AS s` are aliases that keep queries short. The `CAST` calls guard against the ID columns having different data types in the two tables; joins require matching types.

Step 6.2 — LEFT JOIN. Find stations and how many trips started at each, keeping stations with zero trips:

```
SELECT
```

```
  s.name,
  s.status,
  COUNT(t.trip_id) AS trips_started
FROM `bigquery-public-data.austin_bikeshare.bikeshare_stations` AS s
LEFT JOIN `bigquery-public-data.austin_bikeshare.bikeshare_trips` AS t
  ON CAST(t.start_station_id AS STRING) = CAST(s.station_id AS STRING)
GROUP BY s.name, s.status
ORDER BY trips_started ASC
LIMIT 20;
```

Step 6.3 — Common Table Expressions (WITH). CTEs break a query into readable named steps:

```
WITH trips_per_station AS (
  SELECT
    start_station_name,
    COUNT(*) AS trip_count,
    AVG(duration_minutes) AS avg_minutes
  FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`
  GROUP BY start_station_name
)
SELECT *
FROM trips_per_station
WHERE trip_count > 1000
```

```
ORDER BY avg_minutes DESC

LIMIT 10;
```

Step 6.4 — Working with dates.

```
SELECT

  EXTRACT(YEAR FROM start_time) AS trip_year,

  COUNT(*) AS trips

FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`

GROUP BY trip_year

ORDER BY trip_year;
```

Other useful functions: `DATE(start_time)`, `FORMAT_TIMESTAMP('%A', start_time)` (day name), `DATE_DIFF(date1, date2, DAY)`.

Step 6.5 — Save results. After running any query, use the results pane:

- **Save results** → **BigQuery table** to keep the output in `lab_data`.
- **Save results** → **CSV** to download (good for small results).
- **Save** → **Save query** in the editor to keep the SQL itself for reuse.

Or save directly with SQL:

```
CREATE OR REPLACE TABLE `lab_data.trips_by_year` AS

SELECT EXTRACT(YEAR FROM start_time) AS trip_year, COUNT(*) AS trips

FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`

GROUP BY trip_year;
```

> **Location gotcha:** this works only if `lab_data` is in the `US` location, the same as the public dataset. If you get a "dataset not found in location" error, recreate `lab_data` in `US`.

✏️ **Exercise 6:** Using a CTE, find the 5 most popular trip routes (start station → end station pair) by number of trips.

Solution WITH routes AS (

```
SELECT start_station_name, end_station_name, COUNT(*) AS trips

FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`

GROUP BY start_station_name, end_station_name
```

)

```
SELECT *

FROM routes

ORDER BY trips DESC

LIMIT 5;
```

Lab 7: Cost Control and Performance Basics

Goal: Understand what you're charged for and write efficient queries.

BigQuery stores data **by column**. A query is billed for the columns it reads across the whole table, not the rows it returns.

Step 7.1 — Compare scanned bytes. Paste each query into the editor (don't run them yet) and note the "This query will process…" estimate:

```
-- Query A: every column

SELECT *

FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`

LIMIT 10;

-- Query B: two columns
```

```
SELECT trip_id, duration_minutes

FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`

LIMIT 10;
```

Query B scans far less, even though both return 10 rows.

Step 7.2 — Check what a query actually did. Run Query B, then in the results pane open **Job information** (bytes processed and billed) and **Execution details** (time spent per stage).

Step 7.3 — Use caching. Run the same query twice. The second run should show it used **cached results** and processed 0 bytes. Caching applies when the query text and underlying data are unchanged.

Step 7.4 — Set a safety limit. In the editor, click **More** → **Query settings** → under **Advanced options**, set **Maximum bytes billed** (e.g., `1000000000` for 1 GB). Queries that would exceed it fail instead of running.

Step 7.5 — Monitor your usage. Query your own job history with `INFORMATION_SCHEMA` (change `region-us` to match your location):

```
SELECT

  creation_time,

  user_email,

  total_bytes_processed / POW(10, 9) AS gb_processed,

  LEFT(query, 80) AS query_start

FROM `region-us`.INFORMATION_SCHEMA.JOBS_BY_USER

WHERE creation_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY)

ORDER BY creation_time DESC;
```

Cost-saving habits

1. Avoid `SELECT *`. Select only the columns you need.
2. Use **Preview** to look at data instead of `SELECT * ... LIMIT`.
3. Read the byte estimate before clicking Run.

4. Filter on partitioned columns when a table is partitioned (check the table's Details tab for "Partitioned by"); BigQuery then skips irrelevant partitions.
5. Save intermediate results to a small table if you'll query them repeatedly.
6. Set a maximum bytes billed limit while learning.

Lab 8: Clean Up and Next Steps

Step 8.1 — Clean up. Sandbox tables expire on their own, but you can tidy up now:

```
DROP VIEW IF EXISTS `lab_data.cheap_products`;

DROP TABLE IF EXISTS `lab_data.electronics`;
```

To remove everything, click the three-dot menu next to `lab_data` → **Delete**, then type the dataset name to confirm. To remove the whole project, go to **IAM & Admin** → **Settings** → **Shut down**.

Step 8.2 — Challenge exercises. Try these without hints:

1. In `usa_names`, find the name that was most popular in the most different years (rank 1 nationally for gender F).
2. In `shakespeare`, find words that appear in every corpus.
3. In `austin_bikeshare`, find the busiest hour of the day for trips (hint: `EXTRACT(HOUR FROM start_time)`).
4. Load a CSV of your own (e.g., a personal budget or a Kaggle dataset) and write three summary queries against it.
5. Rewrite one of your earlier queries to scan less data, and compare the byte estimates.

Step 8.3 — What to learn next.

- **Window functions:** `ROW_NUMBER()`, `RANK()`, `LAG()`, running totals with `SUM() OVER (...)`.
- **Nested and repeated data:** `STRUCT`, `ARRAY`, and `UNNEST`.
- **Partitioning and clustering** your own tables for performance.
- **Scheduled queries** to automate recurring reports.
- **Looker Studio** to build dashboards on BigQuery data.
- **BigQuery ML** to train simple models with SQL (`CREATE MODEL`).
- **Python access** via the `google-cloud-bigquery` client library or notebooks.

Helpful resources: Google's official BigQuery documentation and SQL reference, the free Google Cloud Skills Boost BigQuery labs, and the public datasets listed in the Google Cloud Marketplace.

Quick Reference

Task	**SQL**
Select columns	`SELECT col1, col2 FROM table`
Filter rows	`WHERE col = 'x' AND col2 > 5`
Sort	`ORDER BY col DESC`
Limit rows returned	`LIMIT 10`
Remove duplicates	`SELECT DISTINCT col`
Aggregate	`COUNT(*)`, `SUM()`, `AVG()`, `MIN()`, `MAX()`
Group	`GROUP BY col`
Filter groups	`HAVING SUM(x) > 100`
Join	`JOIN t2 ON t1.id = t2.id`
Named subquery	`WITH name AS (SELECT ...)`
Create table from query	`CREATE OR REPLACE TABLE ds.t AS SELECT ...`
Create view	`CREATE OR REPLACE VIEW ds.v AS SELECT ...`
Delete table	`DROP TABLE IF EXISTS ds.t`
Change type	`CAST(col AS STRING)`
Extract date part	`EXTRACT(YEAR FROM ts)`